Tokens-to-Trace: What an Agent Must Read Before It Can Change Your Code

Vasili Gavrilov

***Abstract*—Before an AI coding agent can change code safely it must read everything that decides what the code does. We count that reading as tokens-to-trace: for one entry point, the tokens of the smallest set of source regions that determine its behavior, classified by residence: in the project, in library source outside it, or in no file at all, only in the version's conventions. Measured on three language pairs, the plain member of each pair keeps a whole path in 1 to 14 thousand tokens inside the project; the layered member spends 124 to 283 thousand tokens outside it, or keeps part of the deciding text where no repository holds it. In 390 controlled agent runs the layered side cost more turns wherever a task crossed files or conventions, and the only silent failures, wrong rows under status 200, came from a convention no source file states.**

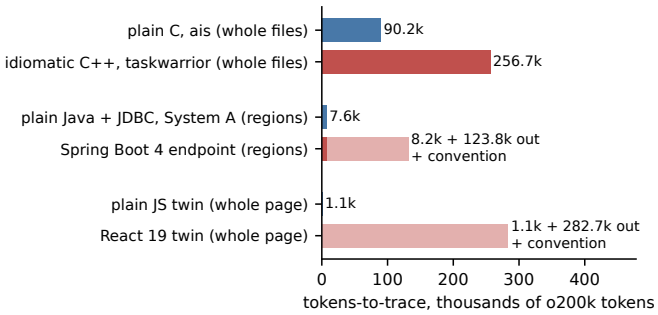

Fig. 1. Tokens-to-trace of one read path per system, each pair at the same counting granularity. Dark bars are text in the project's own repository; light bars are framework source outside it; "conv." marks behavior fixed by version convention, in no file at all.

The reader changed

A class hides a representation and an interface hides an implementation. A framework hides something larger: control flow. Each is a remedy for a limit of human working memory. An agent is not under that limit and is under others: the context it carries and the relationships it must resolve before acting. A hidden implementation is a relationship to resolve, which abstraction adds rather than removes.

The machine cost of abstraction was priced decades ago by compiler work and the zero-overhead principle, and nothing here rests on it. The reader's cost was never priced, and the reader has changed. Studies of agent trajectories already show where the money goes: agents spend most of their tokens reading, and the language with the longest programs is not the one that costs the most [10], [11]. Those are counts of consumption, properties of a model, a harness and a date. This article contributes the static count: a property of the code alone, computable before anything is built or any agent runs. The measure is the contribution; the systems measured below are demonstrations of how to apply it, and the verdicts are theirs, not the measure's.

The measure

Tokens-to-trace of an entry point is the token count of the minimal set of source regions that fully determines its behavior, wire to storage and back: routing, authentication, authorization, handler logic, query, schema, serialization, error mapping. A region is in the set iff removing it leaves some behavior undetermined. A platform floor (JDK, libc, the browser engine, the database) is excluded identically everywhere; anything the application configures, or that varies by library version, is included. Each region is classified by residence:

1) **repository**: read directly from the project's own source;
2) **library source**: text outside the repository that decides behavior before the program runs;
3) **convention**: fixed by defaults of the exact version in use, recoverable from its documentation or a runtime observation, not from any source file.

Tokens are o200k, counted with tiktoken [7]: a public, model-neutral count near what a commercial agent is billed (the vendor's own tokenizer differs, so the window fractions below are approximate). The token count of a region set is exact: anyone can recompute it from the manifests published with the full study [14], and a disagreement is about a manifest row. The set itself is a backward slice in Weiser's sense [8], derived by hand, and a slice is a judgment: a blind second rater agreed on 97 of 103 files of the C++ closure and traced three times the printed Spring category 2, deeper into framework source, so the framework numbers below are conservative lower bounds. Where the floor sits is a declared judgment too, and moving it moves the numbers by a stated factor: 2.5 for the C system below, 2.4 for the framework's category 2; a stricter floor turns the plain sides' zeros into small numbers as well, largest for C. Categories 1 and 2 are counts; category 3 is a classification, its documentation size unmeasured. In the agent's own terms the closure is the needed context: what must be in front of it before it can decide that a change is safe.

Six closures, three pairs

One read path per system, each doing the same kind of work: an authenticated or filtered list over a local store (Figure 1).

C against C++. The plain C system is ais, an associative index of 10,597 code lines written to an explicit style guide [6]; its recall path, argv to the bytes on disk, closes in 16 files. The C++ system is taskwarrior 2.6.2, a widely used task manager and the last all-C++ release of its line [3]; its `task`

`list` closes in 99 files. Whole files on both sides: 90,151 tokens against 256,664, a factor of 2.8 in tokens and 6 in files. At region granularity the C path is 14,444 tokens: 7% of one 200k context window. To bound the author against the style, Monocypher's AEAD encrypt path [5], a library nobody here wrote, closes at 7,560 tokens: at the algorithmic extreme the plain closure is the same size whoever writes it.

Plain Java against Spring. System A is a commercial web application in production, 39,677 code lines, plain Java with JDBC, reported anonymously. Its endpoint (token authentication, role gate, per-object ACL, a 4-table JOIN, streamed JSON, central error mapping) closes in 813 lines across 17 files, 7,566 tokens: 3.8% of a window, with categories 2 and 3 at zero, because every callee is named literally at its call site. The framework subject is a well-regarded Spring Boot 4 implementation of the RealWorld specification [4]. Its comparable endpoint needs 8,175 tokens in the repository, 8% more than System A's whole closure, plus 123,794 tokens of framework classes on the dispatch path, measured from the published sources jars (51,342 under the most generous floor). Beneath that, uncounted, hibernate-core alone is 6.7M tokens of source. And part of what Spring does is written in no file at all: the derived-query grammar that turns `findAllByAuthorProfileUserName` into a join tree, parameter binding, filter-chain order, fetch and flush defaults. That is category 3, and no counting granularity reaches it: a convention that is in no repository cannot be read at any granularity.

Plain JS against React. Behavior-matched twins of the list screen every application has (fetch, filter, sort, select, delete), equal on the graded projection of the rendered DOM. The pages themselves are the same size, 1,092 tokens against 1,145: writing concedes nothing to JSX. Behind the React page sit 282,717 tokens of the runtime's own source over 29 modules (the whole runtime is 641,367), 2.3 times the Spring closure.

The implicit context differs the same way: K&R is 272 pages, Stroustrup 1,368 [1], [2], and a framework's reference moves with the version, so the training corpus superposes versions. Window growth absorbs the C++ row's volume; it does not remove a coherence problem, and long contexts degrade before they fill [9].

390 RUNS: WHAT THE AGENT ACTUALLY PAYS

A count of text is not yet a cost. Four experiments put agents on the measured pairs: 390 runs total, one model (claude-sonnet-5, headless), tasks that name behavior and never a file or a function, graded by hidden tests the agent never saw. A turn is one model request; in every recorded run it equals tool calls plus one, so the metric counts actions and tracks money loosely ($r = 0.70$ with dollars across the 280 pairs runs; cached prefixes make the per-run difference cents). The runs price two of the measure's three residences, file scatter and convention; no task here forced reading the category-2 volume, so the biggest bars of Figure 1 stay unpriced by any run. Table I is the summary; the raw run records ship with the study [14].

The constructs, priced one at a time. Six C/C++ pairs isolate one construct each (a virtual registry, a template, operator overloads, RAII, inheritance, a mix), same behavior, same tasks, hidden tests, 40 runs per pair. Correctness sits at ceiling: 278 of 280 passed, and 2 of 20 against 0 of 20 resolves nothing (Fisher $p = 0.24$). Turns do (Figure 2): the C++ side cost more in all seven pairs, with three effects above the measured rerun noise: the registry spread over eight files ($+3.50$ turns, $p = 0.003$), the mix ($+2.80$, $p < 0.001$), the template ($+1.70$, $p = 0.004$). The control pair is the point: the same registry classes in a single file cost $+0.50$, indistinguishable from zero, so in these pairs the walk across files carries the measurable cost, and any price of the dispatch itself sits below the noise floor. The mechanism fits what an agent does: a callee named at the call site is one search; a registry adds a search before the first can land, finding the registration site; a convention terminates no search at all. And held to identical behavior the C++ text is only 1.17 times the C: the 2.8 between the whole tools is scope and file scatter, not syntax. The experiment ran twice; three C++ sides were repaired between executions, the printed numbers are the second, and the archived first execution shows the same six positive signs. On unchanged pairs the small deltas moved (RAII $+0.50$ then $+1.05$), and the noise band of Figure 2 doubles as an A/A calibration: applied to that null pair, the study's own tests declare significance, so we trust the sign, the three large effects, and nothing of a turn or less.

Experiment	Runs	Pass	Plain	Layered	p
C/C++ constructs, turns	280	278/280	7.87	9.41	0.031^a
Java endpoint, turns	40	38/40	7.15	8.40	0.023^b
Java endpoint, wall s			24.9	30.2	0.022^b
Web twins, turns	40	40/40	6.95	6.85	0.91
Web version traps	30	30/30	0 wrong choices		
its boundary task, turns	15	15/15	6.0	12–13	^{c}

TABLE I

THE RUN EXPERIMENTS. TURNS IS THE PRE-DECLARED PRIMARY METRIC; P-VALUES ARE UNADJUSTED; DOLLAR COST AND CUMULATIVE CONTEXT DO NOT RESOLVE AT THIS n EXCEPT WHERE FOOTNOTED. ^{a}SIGN TEST OVER THE SIX CONSTRUCTS, TWO-SIDED; 0.031 IS THE SMALLEST VALUE SIX PAIRS CAN GIVE. ^{b}PERMUTATION WITHIN TASK, ONE-SIDED, 100,000 RESAMPLES; WALL TIME LARGELY RE-MEASURES TURNS. BOTH ENDPOINT FAILURES ARE THE SPRING SIDE'S SILENT BINDING MISS; THE TWO C/C++ FAILURES ARE ONE C++ CELL MISSING THE SAME EMPTY-STORE EDGE TWICE, LOUDLY. ^{c}FIVE RUNS PER CELL; CUMULATIVE CONTEXT 323 AND 353 THOUSAND TOKENS AGAINST PLAIN'S 181, \$0.14 A RUN AGAINST \$0.09.

The endpoint, two stacks. One endpoint, an item list and item create behind bearer-token authentication over SQL, built twice with the same routes, schema, and observable behavior, equivalence verified at the HTTP boundary before any run. The plain side is one file, JDK `HttpServer` and JDBC, 108 lines; the framework side is seven classes of current Spring idiom (Boot 3.3; the closure above is measured on Boot 4). Four maintenance tasks, five repetitions each (Figure 3). The framework side cost 1.25 turns and 5.3 seconds more per run, an estimate the A/A calibration holds to suggestive, and gave the only silent failures of all 390: twice, on the filter task, the agent wrote `@RequestParam(required = false) Integer minLen` with correct JPQL behind it. Spring derives the wire name from the compiled Java parameter name (every Boot build compiles with `-parameters`; without

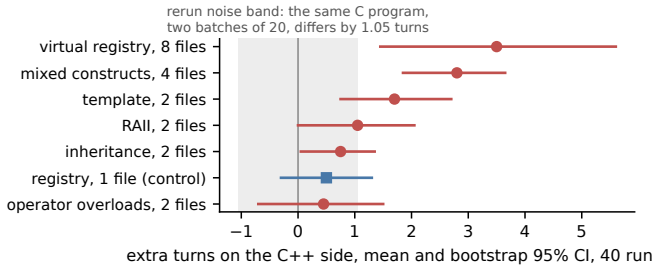

Fig. 2. The C/C++ pairs: extra turns on the C++ side per construct, mean and bootstrap 95% CI (20,000 resamples). The square is the control: the eight-file registry's classes moved into one file. Effect sizes (Cliff's δ) run 0.23 to 0.80, largest for the mix and the scattered registry. Intervals reflect run-to-run variation only; effects inside the gray band are not claims.

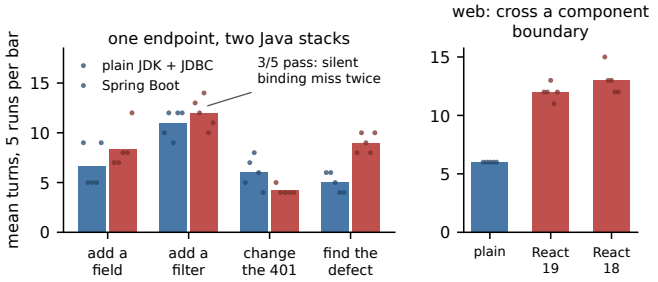

Fig. 3. Left: the Java endpoint, mean turns per task, dots are individual runs. Right: the web pair's boundary task, focusing a child component's input from the parent, against React 19 and React 18.

that flag the same code fails loudly at the request); the URL says `min_len`; the parameter stayed null and the endpoint returned the unfiltered list with status 200. Nothing in the compiler, the runtime or the response flagged it. The deciding rule is a category-3 convention, and this is what its failure looks like: silent wrong rows. The plain side carries the wire name as a literal string in visible parsing code, so the miss has no silent form there, five of five; 2 of 5 against 0 of 5 resolves nothing by count, and the mechanism is the finding. Spring also saturates every training corpus, so these runs price recalling its conventions, not reading its 124k: the density favors the framework side, and the failure is one model's recall on one date. The direction is not uniform, and the exception is instructive: on the 401-contract task Spring was cheaper, 4.2 turns against 6.0, because its filter puts the whole authentication contract in one small class. Where the framework localizes a change it wins the turn count; where the behavior crosses its conventions it loses turns or correctness.

The web pair. On four local maintenance tasks the twins did not separate, and React was cheaper on the add-a-field task, 5.6 turns against 7.6 (five runs a side, not resolving): the component boundary puts that change in one small file. Two tasks whose correct idiom changed between React 18 and 19 produced zero version-wrong choices in the 20 runs on version-pinned sides: this model's habitual idioms are the version-portable ones, and another model's habits need not be. What separated is the boundary: focusing a child component's input from the parent cost twice the turns of the unfactored plain page, every run passing; no factored plain cell ran, so the price belongs to the boundary, not to React specifically. The web pair repeats the C/C++ result, not the Java one: nothing broke, and the agent pays for the walk across the boundary.

THE SUBJECTS ARE DEMONSTRATIONS, NOT SAMPLES

One rater derived the closures, he is the author of the plain systems, and he chose the endpoints and where the floors sit. In a sampling study that is a flaw. This is not a sampling study, and the choices are the method: each plain subject demonstrates how cheap a path becomes when it is built for the new reader, the way a proof of existence picks its witness. The demonstration is open in both directions. Write your path plainer than these and the point strengthens; nobody claims these witnesses are minimal. And you can always find less efficient implementations of either style: a heavier Spring than the measured RealWorld, a more scattered C++ than taskwarrior, and equally a C codebase whose closure measures like taskwarrior's, since nginx-style ops-struct vtables would resolve no more cheaply than a registry. The style is not the language; the closure is the property, and the language only sets how hard the low closure is to reach. The framework bars do travel: categories 2 and 3 are properties of the jars and the version, carried by every application on them.

That inversion is the recommendation. Do not argue about taste: measure the closure of one representative path in a reference implementation of each candidate stack, at a declared floor, and compare numbers. A plain trace costs an afternoon with a tokenizer; a framework trace costs a day and is rater-sensitive; either settles where the deciding text of your candidate lives before the system exists.

The low closure is also reachable on demand. An agent's default is its training-set average, the industry idiom; one line in the prompt, "plain C", "plain Java with JDBC, streaming", "plain vanilla JavaScript", steers it off that mean, and the closures above are the result of that one-line bias. Several of the plain C projects in the study were rewritten by an AI coder under exactly that direction [14]. The remaining human work is the objective (requirements precise enough to implement) and the guards (what the tests must assert); the working practices this compresses are published as recipes [15].

BY COINCIDENCE, IT IS C

The recommendation is not a return to C, and it is not nostalgia. It is: write in a smaller subset of whatever stack you hold, with fewer indirections, still portable. Inside Java that subset is JDBC, visible SQL and streamed rows, and System A shows it runs a commercial product. Inside the browser it is the DOM the engine already gives you, and the twin shows it costs the same tokens to write. Inside C++ it is the part that compiles as C. Take the subtraction to its end, a language whose semantics have not moved in forty years, so that everything ever written about it describes the same language and the training corpus is one coherent body, and

the language you are holding is, by coincidence, C. The old languages were expensive for unaided humans, and the expense was exactly the bookkeeping agents now do.

What of abstraction's other jobs? Reuse of hard, stable code survives fully: that is the platform floor, and the measured plain systems keep it (the database, the cipher, the browser engine). Contracts between teams partially survive as module seams. Compression for small working memory loses its purpose where the whole deciding text fits beside the work, and reliable mechanical edits, the reason for DRY, showed no failure in a dedicated 25-run experiment at up to 30 exact-copy sites, though the near-miss cross-file form remains untested [14].

Limits

The numbers bound mechanisms, not laws. One model ran, on one harness version, on one date; agent cost is a property of the model and the representation together. The cells are five runs, the tasks are fixed by design, so every claim is about these tasks on this model; every run sat far below the context window, so what a closure that outgrows the window costs is untested, and the volume experiment that decides it, cross-module tasks on taskwarrior against a plain twin, is named in the study as open. The measure prices reading and nothing else. It does not price security: framework defaults are amortized defenses (header parsing, encoding, injection), and the plain stack buys that coverage back with tests and review, a cost no closure counts. It does not price what a type system removes, the certification regime that verifies every duplicated site separately, or the onboarding of the humans who share a house style. Prior cross-language work compares pass rates or run-time spend [12], [13]; tokens-to-trace is the pre-run complement, and both counts are needed, because an agent answering from its weights makes missing context look free.

What to do Monday

- Before choosing a stack, measure tokens-to-trace of one representative path in each candidate, at a declared floor, and ask where the deciding text lives. A path whose behavior rests on convention can fail the way the filter task failed, and when it fails, nothing in the toolchain flags it; whether it fails is a property of the model's habits, as the React traps show.
- When writing with an agent, state the bias in the prompt: the plain subset of your stack, named in one line. The agent's default is the industry average; the low closure is one sentence away.
- Keep abstraction where a platform floor earns it, and spend the human effort on the requirements and the tests.
- If your path measures cheaper than the witnesses here, publish the number. The measure is recomputable, and the comparison is the point.

References

[1] B. Kernighan, D. Ritchie. *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[2] B. Stroustrup. *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.

[3] taskwarrior 2.6.2. https://github.com/GothenburgBitFactory/taskwarrior.

[4] `realworld-springboot-java` v2.1.1. https://github.com/raeperd/realworld-springboot-java.

[5] L. Vaillant. Monocypher 4.0.3. https://monocypher.org.

[6] `ais`, an associative index in C99, with its style guide under `doc/dev`. https://github.com/Anode1/ais.

[7] tiktoken, OpenAI's tokenizer library; `o200k_base`. https://github.com/openai/tiktoken.

[8] M. Weiser. *Program Slicing*. IEEE Trans. Software Eng., 1984.

[9] N. F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. TACL, 2024.

[10] *The Best Programming Language for Tokenmaxxing: An Investigation of Coding Agent Behavior Across Programming Languages*. arXiv:2607.22807, 2026.

[11] *How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks*. arXiv:2604.22750, 2026.

[12] F. Cassano et al. *MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation*. IEEE TSE, 2023.

[13] C. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.

[14] V. Gavrilov. *When Abstraction Becomes Indirection: Tokens-to-trace, Measured on Six Stacks*. Zenodo, 2026. https://doi.org/10.5281/zenodo.22113993. Manifests, pair programs, graders and raw run records.

[15] `agent-recipes`: working practices for coding with AI agents. https://github.com/Anode1/agent-recipes.